TP5 – Analizar antes de pedir

Función analizada: `queryTicketRows()`, que se ejecuta en cada `GET /tickets` y trae los tickets desde SQL Server ejecutando `sqlcmd` como proceso externo.

Punto de partida: la función anda. Los tres problemas no son errores de hoy. Son decisiones que quedaron tomadas sin que nadie las decidiera, y que se rompen o se encarecen cuando el sistema crece o cuando lleguen las funcionalidades ya planificadas (crear y editar tickets).

---

1. Tres problemas (del más grave al menos grave)

Problema 1 – La forma de hablar con la base no admite parámetros, y la próxima funcionalidad los necesita

Qué decisión quedó tomada: el acceso a datos es "armar un texto con la consulta y pasárselo a `sqlcmd -Q`". `sqlcmd` no tiene forma de enviar parámetros separados de la consulta. Hoy no importa, porque el `SELECT` es fijo y no recibe nada del usuario.

Escenario concreto: el próximo paso es crear y editar tickets desde el formulario. Quien lo programe va a copiar este patrón, porque es el que ya está en el repositorio, y la única forma de meter el título del formulario en la consulta es concatenarlo:
`-Q "INSERT INTO Tickets (Titulo) VALUES ('" + titulo + "')"`.

- Un usuario carga el ticket **"No anda el login de O'Neil"**. El apóstrofo cierra el texto antes de tiempo, `sqlcmd` falla y el ticket no se guarda. Además, `sqlcmd` sin `-b` devuelve código de salida 0 aunque la consulta falle, así que el backend puede responder "creado" sin que se haya guardado nada.
- Alguien escribe en el título `'); DELETE FROM Tickets; --` y **se borran todos los tickets**: inyección SQL.

Por qué es el más grave: puede destruir o exponer datos, y no es un riesgo lejano: aparece con la funcionalidad que ya está anunciada como la siguiente.

Problema 2 – El JSON se rompe cuando la tabla pasa unos 2 KB

Qué decisión quedó tomada: confiar en que la salida de `sqlcmd` es un JSON limpio en una sola línea. Con `FOR JSON`, SQL Server **parte el resultado en filas de unos 2033 caracteres**, y `sqlcmd` imprime cada fila en una línea nueva. `stdout.trim()` solo limpia los extremos, no los saltos de línea del medio.

Escenario concreto: hoy hay 5 tickets con títulos cortos y todo el JSON entra en una sola fila, por eso anda. Cuando la tabla crezca (por ejemplo, unos 20 tickets con una descripción de un párrafo), el JSON supera los 2033 caracteres y llega cortado en varias líneas. Si un corte cae en medio de un texto, por ejemplo dentro de `"descripcion":"El usuario no puede ...`, `JSON.parse` falla con un error de carácter inválido. Desde ese momento `GET /tickets` **devuelve 500 para todos los usuarios**, y la pantalla de tickets queda vacía de un momento para el otro, el día en que se carga el ticket que hace pasar el límite. Nadie tocó el código y "dejó de andar".

Más adelante aparece un segundo límite: `maxBuffer` de 10 MB. Cuando la tabla completa supere ese tamaño, Node corta el proceso y el GET falla otra vez.

Por qué es el segundo: no destruye datos, pero tira abajo la funcionalidad principal para todos a la vez, y va a pasar solo por uso normal.

Problema 3 – `SELECT *`: el contrato de la API lo decide la tabla, no el backend

Qué decisión quedó tomada: qué datos se mandan al frontend. Con `SELECT *`, la respuesta de `GET /tickets` incluye **todas las columnas que tenga la tabla en ese momento**, y la función devuelve `unknown[]` sin mirar qué hay adentro.

Escenario concreto:

- Alguien agrega a `Tickets` una columna `NotasInternas` para que soporte escriba comentarios que el cliente no debería ver. Sin tocar el backend, esas notas **empiezan a viajar en cada** `GET /tickets` y se pueden ver con F12 en el navegador de cualquier usuario.
- Alguien agrega una columna `Adjunto varbinary(max)` para guardar capturas de pantalla. `FOR JSON` las convierte a base64, y cada carga de la lista pasa a traer **varios MB por ticket**. Además, se llega mucho antes al Problema 2 y al límite de 10 MB.

Por qué es el tercero: hoy no pasa nada, pero cualquier cambio en la base cambia lo que recibe el frontend sin que nadie lo revise.

Otros riesgos detectados (no entran en el top 3)

- **Un proceso nuevo por cada GET:** cada pedido lanza `sqlcmd`, abre una conexión nueva y se autentica. Con una persona no se nota; con 40 personas abriendo la lista a las 9 de la mañana son 40 procesos y 40 conexiones al mismo tiempo, sin reutilizar ninguna.
- `-E` **(autenticación de Windows):** se conecta con el usuario de Windows que está corriendo el backend. Anda en la PC del desarrollador; en un servidor, un contenedor Linux o una cuenta de servicio, falla. Lo mismo pasa si `sqlcmd` no está instalado ahí.
- `DB_DATABASE` **vacío no se valida:** si falta, `sqlcmd` se conecta a la base por defecto del usuario, y el error aparece como "Invalid object name 'Tickets'" en vez de un mensaje claro.

---

2. El prompt

```
Contexto
Tengo un backend en Node.js con TypeScript. La función queryTicketRows() (te la pego
abajo) se ejecuta en cada GET /tickets y trae los tickets de SQL Server lanzando el
programa sqlcmd como proceso externo. Hoy funciona: hay 5 tickets y un solo usuario.
El próximo paso del proyecto es crear y editar tickets desde un formulario.

Problemas que ya detecté y quiero resolver
1. sqlcmd no permite consultas con parámetros. Cuando hagamos el alta y la edición,
   habría que concatenar texto del usuario en la consulta (inyección SQL; un título
   con apóstrofo rompe la consulta).
2. FOR JSON parte el resultado en filas de ~2033 caracteres y sqlcmd las imprime en
   líneas separadas. Cuando la tabla supere ese tamaño, JSON.parse va a fallar.
3. SELECT * hace que cualquier columna nueva de la tabla se mande al frontend.

Qué camino tomar (ya está decidido, no propongas alternativas)
- Reemplazá sqlcmd por el paquete npm "mssql". No uses un ORM ni otra librería.
- Creá UN pool de conexiones a nivel de módulo, la primera vez que se necesite, y
  reutilizalo en las siguientes llamadas. No abras una conexión por pedido.
- Autenticación: usuario y contraseña de SQL Server leídos del .env como DB_USER y
  DB_PASSWORD. Nunca escritos en el código.
- Seguí usando FOR JSON PATH en la consulta, para que el JSON que sale sea exactamente
  el mismo que hoy (mismos nombres de propiedad, mismo formato de fechas, y las
  columnas NULL siguen sin aparecer). Uní todas las filas que devuelve el driver antes
  de hacer JSON.parse.
- Cambiá SELECT * por una lista explícita de columnas. ANTES de escribir código,
  mostrame las columnas de la tabla Tickets y cuáles usa el frontend; la lista la
  elijo yo.
- Dejá el acceso a datos preparado para consultas con parámetros (request.input), pero
  en esta función no hace falta ninguno.
```

```
Qué NO tenés que tocar
- El nombre, los parámetros y el tipo de retorno: queryTicketRows(): Promise<unknown[]>.
- La forma de los datos que devuelve: el mismo array de objetos que hoy.
- Si la tabla está vacía, sigue devolviendo [].
- La carga del .env (misma ruta) y las variables DB_SERVER y DB_DATABASE.
- El error claro cuando falta DB_SERVER. Agregá el mismo control para DB_DATABASE,
  DB_USER y DB_PASSWORD.
- La ruta GET /tickets, el controller y cualquier otro archivo, salvo package.json
  para agregar mssql.
- No agregues paginación, ni caché, ni los endpoints de crear o editar. Eso es para
  después.

Cómo quiero que trabajes
1. Primero: mostrame las columnas (ver arriba) y un plan corto. No escribas código
   hasta que lo apruebe.
2. Después: el código de la función y el cambio en package.json.
3. Al final: explicame cómo comprobar que el comportamiento no cambió:
   - comparar el JSON de GET /tickets de antes y de después con los 5 tickets actuales;
   - probar con un ticket de descripción larga (más de 5000 caracteres) y otro con un
     apóstrofo en el título;
   - qué pasa si la base no está disponible.
Si algo de lo pedido no se puede hacer sin romper una de las restricciones, decímelo en
lugar de resolverlo por tu cuenta.

[código de queryTicketRows]
```

---

3. Justificación del prompt

Qué contexto le puse y por qué

- **Que la función anda y cuántos datos hay hoy.** Si no lo digo, la IA busca un error que no existe, o reescribe todo "por las dudas". Saber que hay 5 tickets y un usuario le explica por qué hoy no falla.
- **Que lo próximo es crear y editar desde un formulario.** Es lo que vuelve grave el Problema 1. Sin ese dato, la IA no tiene motivo para pensar en parámetros, porque este `SELECT` no los usa.
- **Los tres problemas, ya diagnosticados, con su causa.** El análisis lo hice yo. Si le pido "encontrá problemas", la IA elige qué arreglar y cuánto cambiar. Al darle los problemas, la mejora apunta a eso y no a otra cosa.

Qué decidí yo en lugar de dejárselo a la IA

Decisión	Qué elegí	Por qué no lo dejé abierto
Cómo conectarse	Paquete `mssql` con un pool compartido	Si le pregunto, puede proponer un ORM (Prisma, TypeORM) o arreglar el parseo de `sqlcmd` y seguir con el proceso externo. Lo primero es un cambio enorme para una función; lo segundo no resuelve el Problema 1.
Autenticación	Usuario y contraseña de SQL desde `.env`	Mantener `-E` con `mssql` necesita un driver nativo extra y sigue atado al usuario de Windows. Es una decisión de infraestructura y la tomo yo.
Formato del resultado	Seguir con `FOR JSON PATH` y unir las filas	Si la IA cambia a un `SELECT` común, el driver devuelve las fechas como objetos `Date` (cambia el formato y puede correr la zona horaria) y las columnas `NULL` aparecen como `null` en vez de no aparecer. El frontend podría romperse sin que nadie note por qué. Con `FOR JSON` el JSON queda idéntico al de hoy.

Decisión	Qué elegí	Por qué no lo dejé abierto
Qué columnas devolver	Las elijo yo, después de ver la tabla	Qué datos ve el frontend es una decisión del negocio, no de la IA. Por eso le pido que me las muestre primero.
Paginación	Fuera de alcance	Cambiaría el contrato de `GET /tickets` y obligaría a tocar el frontend. Es otra tarea.

Qué límites le puse y por qué

- **La firma y la forma de los datos no cambian.** Así ningún archivo que llame a `queryTicketRows()` tiene que cambiar, y puedo comprobar el resultado comparando el JSON de antes y de después.
- **Un solo archivo (más `package.json`).** Si el cambio se desparrama por el controller y las rutas, no puedo revisarlo bien ni saber qué rompió qué.
- **Nada de crear o editar todavía.** Le pido que deje el patrón con parámetros listo, pero sin implementar esas funcionalidades: primero quiero la base bien hecha y probada.
- **Plan primero, código después.** Así puedo frenarla antes de que tome una decisión que no quiero.
- **"Si no se puede, decímelo".** Si una restricción choca con otra, prefiero enterarme a que la IA la rompa en silencio.
- **Cómo verificar.** Le pido pruebas que reproducen exactamente los escenarios de los Problemas 1 y 2 (apóstrofo y descripción larga), no un "probalo y fijate".